

Recupero e implementazione di componenti ML per analisi dati in un contesto distribuito

Gabriele Stentella
984491

Relatore:
Prof. Claudio A. Ardagna

Correlatori:
Prof. Marco Anisetti
Dott. Antongiaco Polimeno

Nel contesto della moderna elaborazione dei dati, l'intelligenza artificiale (IA) e il cloud computing rappresentano due pilastri fondamentali per la trasformazione digitale delle aziende, delle organizzazioni pubbliche e degli istituti di ricerca. L'IA, attraverso algoritmi avanzati di *machine learning* (ML) e *deep learning* (DL), consente di analizzare grandi quantità di dati in modo efficiente, identificare pattern e trend nascosti, e assistere nelle decisioni basandosi su modelli statistici. Questo approccio permette di ottenere informazioni di valore a partire da dati non strutturati con elevata dimensionalità, eterogeneità e presenza massiccia di correlazioni spurie, detti *big data*.

Il *cloud computing* offre un'infrastruttura flessibile e scalabile per archiviare, elaborare e distribuire i dati necessari per gli algoritmi di IA. Grazie al cloud, è possibile accedere a risorse informatiche *on-demand*, riducendo i costi di gestione e manutenzione dei server fisici. Inoltre, consente di implementare soluzioni che richiedono grande potenza di calcolo in modo rapido ed efficiente, dunque può essere sfruttato per addestrare modelli complessi e gestire carichi di lavoro intensivi.

L'integrazione sinergica tra intelligenza artificiale e cloud computing apre nuove opportunità di sfruttare appieno il potenziale dei propri dati, migliorare la competitività sul mercato e innovare.

Sebbene attraverso il cloud si possa accedere a risorse molto ampie, per svolgere analisi su insiemi di dati ad elevata dimensionalità in modo efficiente, è necessario parallelizzare il carico computazionale. L'obiettivo è quello di dividere sia i dataset sia le operazioni da svolgere sui dati su diversi nodi di un *cluster*, siano essi macchine fisicamente distinte o nodi implementati in un ambiente virtualizzato. Con l'avvento dei big data sono quindi nati dei framework pensati per gestire sia la distribuzione di dati su più dispositivi, sia l'orchestrazione delle operazioni che vengono svolte su uno stesso insieme di dati, ma su nodi diversi. La prima architettura logica sviluppata è stata *Apache Hadoop*, un insieme di software *open-source* che consente di distribuire il carico computazionale con il modello *MapReduce* e distribuire i dati attraverso l'*Hadoop distributed file system* (HDFS). Per superare il processo sequenziale imposto dall'utilizzo di *MapReduce*, è stato sviluppato successivamente *Apache Spark* che consente di leggere i dati dalla memoria, eseguire operazioni e scrivere i risultati in un singolo step. *Apache Spark* ha inoltre introdotto la possibilità di riutilizzo dei dati mantenuti in cache, caratteristica che consente una grande ottimizzazione di algoritmi che chiamano ripetutamente una funzione su uno stesso dataset, come quelli di machine learning. Fra le API offerte da *Spark* si trova infatti una libreria specificatamente dedicata al machine learning (MLlib), ampiamente utilizzata in questo lavoro di tesi. Un ulteriore vantaggio del framework *Spark* è quello di non avere un proprio sistema di archiviazione, ma di permettere l'utilizzo sia di sistemi esterni come HDFS o *Amazon S3*, sia di soluzioni personalizzate.

Gli algoritmi di ML possono essere utilizzati sia come strumento predittivo, ovvero per cercare di generalizzare le proprietà osservate in una collezione di dati, sia come strumento per effettuare il *data mining*, quindi per ricercare proprietà non note all'interno dei dati che si hanno a disposizione. In generale, l'utilizzo di intelligenza artificiale nell'analisi dati mira a trovare delle correlazioni fra le variabili presenti nella collezione di dati, con il fine di renderle evidenti ad un'analista oppure di utilizzarle per costruire un modello predittivo.

L'obiettivo di questa tesi è quello di recuperare algoritmi di machine learning per l'analisi dati sviluppati come *Spark tasks* in linguaggio Java, e studiare l'implementazione di nuovi algoritmi seguendo un approccio

più adatto a modelli di DL, quindi utilizzando il linguaggio *Python* e la libreria *Tensorflow*; l'esecuzione avviene in un contesto distribuito gestito dall'orchestratore *Kubernetes*.

La motivazione del lavoro è da vedersi nell'evidente vantaggio che deriva dalla possibilità di disaccoppiare il codice dall'infrastruttura su cui viene eseguito. Attraverso il recupero di algoritmi pre-esistenti e l'implementazione di nuovi, utilizzando tecnologie differenti, si è studiata la possibilità di eseguire componenti software su un'infrastruttura distribuita e diversa da quella per cui sono state originariamente pensate. Un aspetto fondamentale è da ricercarsi nelle minime modifiche operate sul codice, talvolta del tutto assenti.

Il lavoro di recupero degli algoritmi è stato portato avanti parallelamente allo studio e all'implementazione di nuovi algoritmi, e l'attività svolta durante il tirocinio può essere riassunta dalle fasi seguenti.

- Analisi degli algoritmi esistenti. È stata effettuata un'analisi dei diversi algoritmi già implementati al fine di individuare quelli più interessanti per l'analisi dei dati. Successivamente ogni algoritmo selezionato è stato analizzato in dettaglio per individuarne le dipendenze e le versioni delle diverse tecnologie utilizzate.
- Aggiornamento e riorganizzazione degli algoritmi. Gli algoritmi selezionati sono stati aggiornati per utilizzare la versione più moderna di *Spark*. Si è anche reso necessario l'aggiornamento di una libreria esterna non più mantenuta, ma utilizzata dagli algoritmi. La struttura dei progetti dei singoli task è stata ripensata in modo tale da poter costruire i diversi *file archivio* (jar), rispettando le eventuali dipendenze presenti fra alcuni task.
- Implementazione di un algoritmo di test in Tensorflow. È stato sviluppato un modello predittivo di *machine learning* in *Python* utilizzando la libreria *Tensorflow*.
- Esecuzione distribuita degli algoritmi. I task sono stati modificati in modo tale da consentirne l'esecuzione su un cluster *Kubernetes*. In seguito sono stati eseguiti su un cluster appositamente creato per il testing, virtualizzato su una singola macchina. Infine è stato implementato un secondo cluster di testing in cloud utilizzando le tecnologie *Amazon EMR on EKS* e *Amazon S3*.
- Test di performance dell'esecuzione distribuita. Sono stati analizzati i dati di log dei task *Spark* e quelli dell'allenamento del modello sviluppato con *Tensorflow*, al fine di dimostrare l'effettiva distribuzione del carico computazionale e dunque i vantaggi dell'esecuzione in contesto distribuito.
- Esecuzione sull'infrastruttura MUSA. Il *deployment* finale è stato eseguito sfruttando l'ambiente distribuito gestito con *Kubernetes*, all'interno dell'infrastruttura sviluppata nell'ambito del progetto MUSA, finanziato con i fondi del PNRR da parte del Ministero dell'Università e della Ricerca.

La tesi lascia spazio per lavori futuri. Primo fra tutti, l'utilizzo di un sistema di *scheduling* dei flussi di lavoro come *Apache Oozie* o *Airflow* per creare delle *pipeline* di task in modo tale da costruire un'infrastruttura idonea ad analisi dati più complessa. In secondo luogo, è da notare che l'approccio proposto per l'implementazione di nuovi algoritmi utilizzando *Python* e *Tensorflow* è scalabile, quindi resta aperta la possibilità di sviluppare nuovi modelli per l'analisi dati, o rendere più efficienti algoritmi già implementati e noti in letteratura.